

Exact KLA with Diagonal Dynamics: Objective and a Chunkwise Algorithm

August 11, 2026

1 Objective

The objective is to compute the Kalman gains of the exact Kalman Delta Network without a token-by-token dense covariance recurrence. We use the notation of the Exact Kalman Delta Network formulation in the main paper. In particular,

$$\mathbf{D}_t = \text{diag}(\boldsymbol{\alpha}_t), \quad \boldsymbol{\Omega}_t = \text{diag}(\boldsymbol{\omega}_t), \quad (1)$$

and “diagonal” refers only to the transition and process-noise covariance. The posterior covariance $\mathbf{P}_t$ remains dense.

Throughout this note, r_t denotes the effective observation-noise variance that appears in the Kalman recurrence. The current implementation optionally parameterizes a raw variance and passes r_t^{raw}/d_k to the filter when `dk_calibration=True`; in that case all formulas below use $r_t = r_t^{\text{raw}}/d_k$.

Given $\mathbf{S}_{t-1}$ and $\mathbf{P}_{t-1}$, ExactKLA runs the recurrence

$$\begin{aligned} \text{predict: } \hat{\mathbf{S}}_t &= \mathbf{D}_t \mathbf{S}_{t-1}, & \hat{\mathbf{P}}_t &= \mathbf{D}_t \mathbf{P}_{t-1} \mathbf{D}_t^\top + \boldsymbol{\Omega}_t, \\ \text{gain: } \boldsymbol{\kappa}_t &= \frac{\hat{\mathbf{P}}_t \mathbf{k}_t}{r_t + \mathbf{k}_t^\top \hat{\mathbf{P}}_t \mathbf{k}_t}, \\ \text{update: } \mathbf{S}_t &= \hat{\mathbf{S}}_t + \boldsymbol{\kappa}_t (\mathbf{v}_t - \hat{\mathbf{S}}_t^\top \mathbf{k}_t)^\top, & \mathbf{P}_t &= (\mathbf{I} - \boldsymbol{\kappa}_t \mathbf{k}_t^\top) \hat{\mathbf{P}}_t, \\ \text{read: } \mathbf{o}_t &= \mathbf{S}_t^\top \mathbf{q}_t. \end{aligned} \quad (2)$$

The layer is initialized with $\mathbf{S}_0 = \mathbf{0}$ and $\mathbf{P}_0 = \mu^{-1} \mathbf{I}$. At later chunk boundaries, $\mathbf{P}_0$ denotes the dense posterior covariance passed from the preceding chunk.

Equivalently, the covariance update can be written

$$\mathbf{P}_t = \hat{\mathbf{P}}_t - \frac{\hat{\mathbf{P}}_t \mathbf{k}_t \mathbf{k}_t^\top \hat{\mathbf{P}}_t}{r_t + \mathbf{k}_t^\top \hat{\mathbf{P}}_t \mathbf{k}_t}. \quad (3)$$

This costs $O(d_k^2)$ arithmetic per token and has a length- T sequential dependency. We seek an algebraically equivalent chunk algorithm that retains the dense covariance at chunk boundaries but computes all gains within a chunk using matrix multiplications and triangular solves.

2 Proposed Chunkwise Innovation Factorization

Consider a chunk of length C . The compact factorization assumes every entry of α_t is strictly positive, as in the model’s exponential/sigmoid gate parameterization; the tokenwise ExactKLA recurrence itself does not require this assumption. Define the elementwise cumulative transition and the covariance contributed by process noise since the chunk boundary:

$$\mathbf{f}_t = \bigodot_{u=1}^t \alpha_u, \quad \mathbf{f}_0 = \mathbf{1}, \quad (4)$$

$$\mathbf{z}_t = \alpha_t^2 \odot \mathbf{z}_{t-1} + \boldsymbol{\omega}_t, \quad \mathbf{z}_0 = \mathbf{0}. \quad (5)$$

The vector $\mathbf{z}_t$ is a chunk-local bookkeeping variable. It is the diagonal covariance contributed by all process-noise terms injected since the beginning of the current chunk, after each term has been propagated through the later diagonal transitions. Explicitly,

$$\mathbf{z}_t = \sum_{s=1}^t \left(\bigodot_{u=s+1}^t \alpha_u^2 \right) \odot \boldsymbol{\omega}_s, \quad (6)$$

where the empty product for $s = t$ is $\mathbf{1}$. Thus $\boldsymbol{\omega}_s$ is added at position s , then attenuated by every transition from $s + 1$ through t . The variable $\mathbf{z}_t$ is not an additional ExactKLA model state and is reset to $\mathbf{0}$ at every chunk boundary; uncertainty inherited from earlier chunks is already contained in the dense boundary covariance $\mathbf{P}_0$.

For the covariance derivation, fix a value coordinate i . Following the main paper, let $\mathbf{S}_t^*$ denote the random latent associative map and let $\mathbf{S}_{t,i}^* \in \mathbb{R}^{d_k}$ denote its i -th column. In contrast, $\mathbf{S}_t$ without a star is the filter’s posterior-mean estimate. Let $v_{t,i}$ denote coordinate i of the observed value vector $\mathbf{v}_t$. The corresponding scalar observation model is

$$v_{t,i} = \mathbf{k}_t^\top \mathbf{S}_{t,i}^* + e_{t,i}, \quad \text{Var}(e_{t,i}) = r_t. \quad (7)$$

The key-space covariance $\mathbf{P}_t$ and gain $\boldsymbol{\kappa}_t$ are shared across all value coordinates i . We therefore derive them using one fixed coordinate and suppress i wherever the result does not depend on it. Before conditioning on any value observation inside the chunk, the marginal covariance at position t is

$$\boldsymbol{\Sigma}_t = \text{diag}(\mathbf{f}_t) \mathbf{P}_0 \text{diag}(\mathbf{f}_t) + \text{diag}(\mathbf{z}_t). \quad (8)$$

Introduce two vectors per token,

$$\mathbf{h}_t = \mathbf{f}_t \odot \mathbf{k}_t, \quad (9)$$

$$\mathbf{w}_t = \mathbf{P}_0 \mathbf{h}_t + (\mathbf{z}_t \oslash \mathbf{f}_t) \odot \mathbf{k}_t, \quad (10)$$

where $\oslash$ denotes elementwise division. For $t \geq s$, diagonal dynamics give

$$\text{Cov}(\mathbf{S}_{t,i}^*, v_{s,i}) = \mathbf{f}_t \odot \mathbf{w}_s, \quad (11)$$

where covariance between a vector and a scalar is the vector

$$\text{Cov}(\mathbf{S}_{t,i}^*, v_{s,i}) = \mathbb{E}[(\mathbf{S}_{t,i}^* - \mathbb{E}\mathbf{S}_{t,i}^*)(v_{s,i} - \mathbb{E}v_{s,i})] \in \mathbb{R}^{d_k}. \quad (12)$$

Indeed, using the scalar observation model and independence of $e_{s,i}$,

$$\text{Cov}(\mathbf{S}_{t,i}^*, v_{s,i}) = \text{Cov}(\mathbf{S}_{t,i}^*, \mathbf{S}_{s,i}^*) \mathbf{k}_s = \mathbf{f}_t \odot \mathbf{w}_s. \quad (13)$$

Consequently, for $t > s$,

$$\text{Cov}(v_{t,i}, v_{s,i}) = \mathbf{h}_t^\top \mathbf{w}_s. \quad (14)$$

Let $\mathbf{H}, \mathbf{W} \in \mathbb{R}^{C \times d_k}$ contain $\mathbf{h}_t^\top$ and $\mathbf{w}_t^\top$ as rows. Construct the symmetric covariance of the chunk's scalar observations $(v_{1,i}, \dots, v_{C,i})$, $\mathbf{G} \in \mathbb{R}^{C \times C}$ from

$$G_{ts} = \begin{cases} \mathbf{h}_t^\top \mathbf{w}_s, & t > s, \\ \mathbf{h}_t^\top \mathbf{w}_t + r_t, & t = s, \\ G_{st}, & t < s. \end{cases} \quad (15)$$

Thus its lower triangle is obtained from the matrix product $\mathbf{H}\mathbf{W}^\top$. Compute the unit-lower-triangular factorization

$$\mathbf{G} = \mathbf{L}\mathbf{R}\mathbf{L}^\top \quad (16)$$

and solve

$$\mathbf{E} = \mathbf{L}^{-1}\mathbf{W}. \quad (17)$$

Here $\mathbf{R}$ is diagonal; a distinct symbol is used because $\mathbf{D}_t$ already denotes the memory transition. The entries R_{tt} are the sequential innovation variances. The proposed formula for all exact gains in the chunk is

$$\boxed{\kappa_t = \frac{\mathbf{f}_t \odot \mathbf{e}_t}{R_{tt}}}, \quad (18)$$

where $\mathbf{e}_t^\top$ is row t of $\mathbf{E}$.

Finally, the dense posterior covariance passed to the next chunk is

$$\mathbf{P}_C = \Sigma_C - \text{diag}(\mathbf{f}_C) \mathbf{E}^\top \mathbf{R}^{-1} \mathbf{E} \text{diag}(\mathbf{f}_C). \quad (19)$$

For implementation, it is preferable not to form $\mathbf{L}$, $\mathbf{R}$, or $\mathbf{E}$ explicitly. Let

$$\mathbf{G} = \mathbf{C}\mathbf{C}^\top, \quad \mathbf{Y} = \mathbf{C}^{-1}\mathbf{W}, \quad (20)$$

where $\mathbf{C}$ is the lower Cholesky factor, and let $\mathbf{y}_t^\top$ be row t of $\mathbf{Y}$. The equivalent formulas are

$$\boxed{\kappa_t = \frac{\mathbf{f}_t \odot \mathbf{y}_t}{C_{tt}}, \quad \mathbf{P}_C = \Sigma_C - \text{diag}(\mathbf{f}_C) \mathbf{Y}^\top \mathbf{Y} \text{diag}(\mathbf{f}_C)}. \quad (21)$$

This form uses one triangular solve, divides by one Cholesky pivot rather than an innovation variance, and expresses the boundary update as a Gram matrix multiplication. This formulation is intended to replace the tokenwise covariance update only within each chunk. Chunks remain sequential through the dense boundary covariance $\mathbf{P}_0$. Verification of the gain identity, numerical behavior, and hardware cost is deliberately left for the next stage.

3 Proposed Implementation Plan

Development should follow a correctness ladder. The production ExactKLA implementation remains unchanged until the chunk algorithm is correct, differentiable, and faster than the current exact gain kernel. Initial development therefore lives under `sandboxes/exact_kla_innovation/`; only the final verified implementation is moved into `lit_gpt/kla_ops/`.

3.1 Stage 1: Freeze the Recurrence Oracle and Baseline

Use `naive_recurrent_exact_kla` in `lit_gpt/kla_ops/exact_naive.py` as the FP64 correctness oracle. Add a small test driver that returns both the gain sequence and the final dense covariance, without changing its recurrence. Before implementing the new algorithm, benchmark the current `triton_fused` ExactKLA path at its currently supported $d_k = 64$, sequence lengths $T \in \{512, 2048, 4096\}$, and several realistic batch-head products. The current fused backend does not support $d_k = 128$, so use the recurrent ExactKLA path as the correctness and performance baseline there. Record gain-only, recurrence-core, and full-mixer forward latency, forward-backward latency, peak memory, and kernel-launch count. Report matched DiagKLA measurements at both $d_k = 64$ and the production $d_k = 128$ geometry.

3.2 Stage 2: Implement the Chunked FP64 Reference

Implement a pure-PyTorch function `exact_kla_gains_innovation_ref` in the sandbox. For each chunk it performs the following operations:

1. Compute $\mathbf{f}_t$ and $\mathbf{z}_t$ for every position in the chunk.
2. Form $\mathbf{H}$ and $\mathbf{W}$, then construct the lower triangle of $\mathbf{G}$ from $\mathbf{H}\mathbf{W}^\top$, add r_t to the diagonal, and copy by symmetry.
3. Compute the Cholesky factor $\mathbf{C}\mathbf{C}^\top = \mathbf{G}$.
4. Solve $\mathbf{C}\mathbf{Y} = \mathbf{W}$, emit $\boldsymbol{\kappa}_t = (\mathbf{f}_t \odot \mathbf{y}_t)/C_{tt}$, and compute $\mathbf{P}_C = \boldsymbol{\Sigma}_C - \text{diag}(\mathbf{f}_C)\mathbf{Y}^\top\mathbf{Y}\text{diag}(\mathbf{f}_C)$.
5. Pass $\mathbf{P}_C$ to the next chunk and repeat until the sequence is complete.

The reference uses FP64 and no diagonal jitter. If $\mathbf{G}$ is not positive definite, the test must fail rather than silently changing the filter.

Forward tests compare every $\boldsymbol{\kappa}_t$ and every chunk-boundary covariance against the tokenwise oracle. Run with `dk_calibration` both enabled and disabled, passing the same effective noise to both algorithms. Cover random positive-definite and highly anisotropic $\mathbf{P}_0$, $\boldsymbol{\alpha}$ near zero and one, tiny and large r_t and $\boldsymbol{\omega}_t$, repeated and nearly collinear keys, multiple batch-head layouts, chunk sizes 1, 2, 4, 8, 16, 32, 64, 128, final partial chunks, and $d_k \in \{64, 128\}$. Check maximum absolute and relative error, covariance symmetry and positive semidefiniteness, the normalized Cholesky residual, minimum Cholesky pivot, and minimum innovation variance. The ordinary-scale FP64 target is maximum absolute error below 10^{-10} ; scaled stress cases use combined absolute and relative tolerances.

3.3 Stage 3: Verify Differentiation

Differentiate a scalar loss depending on both the full gain sequence and $\mathbf{P}_T$. Compare the chunk reference with the recurrent oracle separately for gradients with respect to $\mathbf{k}$, $\boldsymbol{\alpha}$, $\boldsymbol{\omega}$, effective r , and the initial covariance $\mathbf{P}_0$. Run PyTorch `gradcheck` on small FP64 cases, then run larger direct gradient comparisons. Also test the chain-rule gradients of the production parameterization with respect to raw r and μ . Next, feed the gains through the existing independent-gain Kalman memory kernel and compare complete $(\mathbf{o}, \mathbf{S}_T, \mathbf{P}_T)$ outputs and gradients with respect to $\mathbf{q}, \mathbf{k}, \mathbf{v}, \boldsymbol{\alpha}, \boldsymbol{\omega}, r, \mu$, and supported initial states. Finally, compare full-layer input and parameter gradients against recurrent mode. No performance implementation begins until each gradient passes independently.

3.4 Stage 4: Build and Profile a Tensorized GPU Prototype

Implement a batched PyTorch CUDA prototype using matrix multiplication, batched Cholesky, and triangular solves. Keep chunks sequential, but batch every head and sequence in the local batch. Profile the following regions separately:

1. construction of $\mathbf{f}, \mathbf{z}, \mathbf{H}, \mathbf{W}$;
2. construction of $\mathbf{G}$;
3. Cholesky/ LDL^T factorization;
4. triangular solve and gain emission;
5. the rank- C boundary covariance update.

Sweep $C \in \{16, 32, 64, 128\}$. At $d_k = 64$, use interleaved CUDA-event timing against the current `triton.fused` composition (`gain_recurrent` plus `chunk_kalman`); at $d_k = 128$, compare against recurrent ExactKLA until a fused incumbent exists. Record achieved FLOP/s, factorization occupancy, dependent launch count, and CUDA-graph/fusion opportunities. The generic prototype is a decomposition and roofline probe, not a strict go/no-go gate: if launch overhead or the small-matrix solver dominates, implement one bounded custom microkernel prototype before deciding whether the design is viable.

3.5 Stage 5: Implement the GPU Forward Path

Use optimized GEMM primitives for large matrix products and write custom kernels only for operations that the profile shows are limiting. Tensor-core acceleration is contingent on a later TF32/BF16 accuracy experiment. The intended decomposition is:

1. a fused chunk-preparation kernel for $\mathbf{f}, \mathbf{z}$, and elementwise scaling;
2. grouped GEMMs for $\mathbf{P}_0 \mathbf{H}^T$, $\mathbf{H} \mathbf{W}^T$, and the boundary covariance downdate;
3. a batched blocked Cholesky or LDL^T kernel for small $C \times C$ matrices;
4. a fused triangular-solve and gain-emission kernel.

The direct boundary formula subtracts two positive-semidefinite matrices and may lose symmetry or positive semidefiniteness through cancellation. Prototype both this formula and a square-root/QR boundary update that propagates a covariance factor rather than forming the downdate by subtraction. Neither path may silently clamp eigenvalues. Record symmetry error, smallest eigenvalue, and the difference from the recurrent Joseph-form update after every chunk. Compute $\mathbf{P}_0$, $\mathbf{G}$, Cholesky, triangular solves, and the covariance downdate using IEEE FP32 initially, with TF32 disabled for these operations. Cast only the emitted gain to the layer’s contract dtype. Compare every forward component against the FP64 reference and recurrent FP32 implementation over $T \in \{512, 2048, 4096\}$. Before implementation, freeze quantitative tolerances for gain, layer output, every gradient, symmetry, and minimum eigenvalue using the current recurrent FP32 path. TF32 or BF16 tensor-core inputs are later opt-in experiments, not assumed-safe defaults.

3.6 Stage 6: Implement the Backward Path

First use automatic differentiation through the chunked PyTorch implementation as the gradient oracle. The optimized backward should checkpoint only chunk-boundary covariances and recompute $\mathbf{f}, \mathbf{z}, \mathbf{H}, \mathbf{W}, \mathbf{G}, \mathbf{C}, \mathbf{Y}$ within each chunk. Process chunks in reverse order while carrying the adjoint of the boundary covariance. Explicitly compare three storage policies: stash every $O((T/C)d_k^2)$ boundary covariance, hierarchical boundary checkpointing, and re-forward from the initial covariance. Report bytes per layer and across the full model for $d_k = 64$ and 128, and stress Cholesky gradients near repeated or small pivots. Validate each input gradient separately against the FP64 oracle before measuring speed or memory.

3.7 Stage 7: Numerical-Stability Alternative

The compact representation contains $\mathbf{z}_t \odot \mathbf{f}_t$, which can overflow when a long chunk has very small cumulative decay. Implement the division-free cross-covariance recurrence as an independent reference path:

$$\mathbf{b}_{s,s} = \Sigma_s \mathbf{k}_s, \quad \mathbf{b}_{t,s} = \alpha_t \odot \mathbf{b}_{t-1,s}, \quad G_{ts} = \mathbf{k}_t^\top \mathbf{b}_{t,s} \quad (t > s), \quad G_{tt} = \mathbf{k}_t^\top \mathbf{b}_{t,t} + r_t, \quad (22)$$

with the upper triangle supplied by symmetry. After factoring $\mathbf{G} = \mathbf{LRL}^\top$, compute gains without $\mathbf{W}$ from

$$\kappa_t = \frac{1}{R_{tt}} \sum_{s=1}^t (L^{-1})_{ts} \mathbf{b}_{t,s}. \quad (23)$$

For the boundary covariance, stack $\mathbf{B}_C = [\mathbf{b}_{C,1}, \dots, \mathbf{b}_{C,C}]$ and use

$$\mathbf{P}_C = \Sigma_C - \mathbf{B}_C \mathbf{G}^{-1} \mathbf{B}_C^\top. \quad (24)$$

Use this complete $O(C^2 d_k)$ path to determine safe chunk lengths for the compact path. Compute a log-prefix range bound before forming $\mathbf{z}_t \odot \mathbf{f}_t$, including subnormal and heterogeneous gate tests. The FP64 reference uses no repair. For production FP32, specify and test an explicit failure policy: retry with the most accurate available reductions, then fall back to a numerically safer exact path. Any jitter, clipping, projection, or renormalization must be reported as a numerical modification and gated against the recurrent oracle.

3.8 Stage 8: Acceptance and Integration

Introduce the implementation first as a distinct constructor mode `mode=innovation_chunk` in `lit_gpt/exact_kla.py`; retain `mode=recurrent` and `mode=triton_fused` as explicit fallbacks rather than adding an implicit environment override. The gain/op API preserves input layouts, effective-noise calibration, optional initial $\mathbf{P}_0$, and optional final $\mathbf{P}_T$. Test these op-level state contracts with arbitrary positive-definite $\mathbf{P}_0$. The mixer layer preserves its existing training-only, no-cache interface and initializes $\mathbf{P}_0 = \mu^{-1} \mathbf{I}$. Integration proceeds only after all of the following conditions hold:

1. FP64 forward and gradient tests match the recurrent oracle.
2. FP32 stress tests remain stable for the training ranges of α , ω , and r .
3. Gain-only latency is reported against `gain_recurrent`; recurrence-core and full mixer latency are reported separately. The primary research target is full-mixer parity with the current

DiagKLA implementation on H200: ExactKLA forward and forward–backward latency must each be no more than $1.1\times$ the corresponding DiagKLA latency at the same batch size, sequence length, head count, and d_k . At the existing $T \approx 2048$, $d_k = 64$ benchmark point, this requires approximately an $8.3\times$ forward speedup and a $4.3\times$ forward–backward speedup over the current ExactKLA backend. Beating the current ExactKLA backend by a smaller margin is an intermediate milestone, not the final acceptance criterion. Report separate pass/fail rows for the matched synthetic $d_k = 64$ geometry and the selected production 220M and 750M local-batch/head geometries at $d_k = 128$; parity must hold at both head dimensions.

4. Peak training memory is measured and documented for both checkpointed and stashed factors.
5. Interleaved benchmarks report performance relative to both current ExactKLA and DiagKLA.

Retain the recurrent and current fused constructor modes until the new path has passed end-to-end layer tests and a short training smoke test.

A Derivation of the Exact Chunk Algorithm

This appendix proves that the chunkwise innovation factorization produces exactly the same gains and chunk-boundary covariance as the tokenwise ExactKLA recurrence. All equalities in this appendix are algebraic equalities in exact arithmetic.

A.1 Conditioning at the Chunk Boundary

Fix a chunk containing positions $1, \dots, C$, and let $\mathcal{F}_0$ denote all observations before the chunk. For one value coordinate i , define the latent key-space state

$$\mathbf{x}_t := \mathbf{S}_{t,i}^* \in \mathbb{R}^{d_k}. \quad (25)$$

Conditioned on $\mathcal{F}_0$, the chunk-entry state has mean $\mathbf{S}_{0,i}$ and covariance

$$\text{Cov}(\mathbf{x}_0 \mid \mathcal{F}_0) = \mathbf{P}_0. \quad (26)$$

Inside the chunk, the latent dynamics and scalar observation are

$$\mathbf{x}_t = \mathbf{D}_t \mathbf{x}_{t-1} + \boldsymbol{\epsilon}_t, \quad \text{Cov}(\boldsymbol{\epsilon}_t) = \boldsymbol{\Omega}_t, \quad (27)$$

$$v_{t,i} = \mathbf{k}_t^\top \mathbf{x}_t + e_{t,i}, \quad \text{Var}(e_{t,i}) = r_t. \quad (28)$$

The process noises, observation noises, and chunk-entry state are mutually independent after conditioning on $\mathcal{F}_0$. We suppress this conditioning below. Means do not affect any covariance calculation, so all latent states and observations may equivalently be centered.

For $t \geq s$, define the cumulative transition

$$\boldsymbol{\Phi}_{t:s} := \mathbf{D}_t \mathbf{D}_{t-1} \cdots \mathbf{D}_s, \quad \boldsymbol{\Phi}_{t:t+1} := \mathbf{I}. \quad (29)$$

Because every $\mathbf{D}_t = \text{diag}(\boldsymbol{\alpha}_t)$ is diagonal, these matrices commute and

$$\boldsymbol{\Phi}_{t:1} = \text{diag}(\mathbf{f}_t), \quad \mathbf{f}_t = \bigodot_{u=1}^t \boldsymbol{\alpha}_u. \quad (30)$$

A.2 Unconditioned Covariance Within a Chunk

Here “unconditioned” means that we condition on $\mathcal{F}_0$, but not yet on observations $v_{1,i}, \dots, v_{t,i}$ from the current chunk. Unrolling the dynamics gives

$$\mathbf{x}_t = \Phi_{t:1} \mathbf{x}_0 + \sum_{s=1}^t \Phi_{t:s+1} \epsilon_s. \quad (31)$$

Independence of the process-noise terms therefore gives

$$\begin{aligned} \Sigma_t &:= \text{Cov}(\mathbf{x}_t) = \Phi_{t:1} P_0 \Phi_{t:1}^\top + \sum_{s=1}^t \Phi_{t:s+1} \Omega_s \Phi_{t:s+1}^\top \\ &= \text{diag}(\mathbf{f}_t) P_0 \text{diag}(\mathbf{f}_t) + \text{diag}(\mathbf{z}_t), \end{aligned} \quad (32)$$

where

$$\mathbf{z}_t = \sum_{s=1}^t \left(\bigodot_{u=s+1}^t \alpha_u^2 \right) \odot \omega_s. \quad (33)$$

Separating the last term of this sum proves the recurrence

$$\mathbf{z}_t = \alpha_t^2 \odot \mathbf{z}_{t-1} + \omega_t, \quad \mathbf{z}_0 = \mathbf{0}. \quad (34)$$

For $t \geq s$, split the state at position s :

$$\mathbf{x}_t = \Phi_{t:s+1} \mathbf{x}_s + \sum_{u=s+1}^t \Phi_{t:u+1} \epsilon_u. \quad (35)$$

The second term is independent of $\mathbf{x}_s$, hence

$$\text{Cov}(\mathbf{x}_t, \mathbf{x}_s) = \Phi_{t:s+1} \Sigma_s. \quad (36)$$

Since the entries of α_t are positive, elementwise division by $\mathbf{f}_s$ is well-defined and

$$\Phi_{t:s+1} = \text{diag}(\mathbf{f}_t \oslash \mathbf{f}_s). \quad (37)$$

A.3 Compact Cross-Covariance Factors

Define

$$\mathbf{h}_s = \mathbf{f}_s \odot \mathbf{k}_s, \quad \mathbf{w}_s = P_0 \mathbf{h}_s + (\mathbf{z}_s \oslash \mathbf{f}_s) \odot \mathbf{k}_s. \quad (38)$$

First observe that

$$\begin{aligned} \Sigma_s \mathbf{k}_s &= \text{diag}(\mathbf{f}_s) P_0 \text{diag}(\mathbf{f}_s) \mathbf{k}_s + \text{diag}(\mathbf{z}_s) \mathbf{k}_s \\ &= \mathbf{f}_s \odot [P_0 \mathbf{h}_s + (\mathbf{z}_s \oslash \mathbf{f}_s) \odot \mathbf{k}_s] \\ &= \mathbf{f}_s \odot \mathbf{w}_s. \end{aligned} \quad (39)$$

The observation noise $e_{s,i}$ is independent of $\mathbf{x}_t$, so for $t \geq s$,

$$\begin{aligned} \text{Cov}(\mathbf{x}_t, v_{s,i}) &= \text{Cov}(\mathbf{x}_t, \mathbf{x}_s) \mathbf{k}_s \\ &= \text{diag}(\mathbf{f}_t \oslash \mathbf{f}_s) \Sigma_s \mathbf{k}_s \\ &= \text{diag}(\mathbf{f}_t \oslash \mathbf{f}_s) (\mathbf{f}_s \odot \mathbf{w}_s) \\ &= \mathbf{f}_t \odot \mathbf{w}_s. \end{aligned} \quad (40)$$

Applying the observation map at position t yields, for $t > s$,

$$\text{Cov}(v_{t,i}, v_{s,i}) = \mathbf{k}_t^\top (\mathbf{f}_t \odot \mathbf{w}_s) = \mathbf{h}_t^\top \mathbf{w}_s. \quad (41)$$

On the diagonal, the independent observation noise contributes r_t :

$$\text{Var}(v_{t,i}) = \mathbf{h}_t^\top \mathbf{w}_t + r_t. \quad (42)$$

Consequently, if $\mathbf{H}$ and $\mathbf{W}$ stack $\mathbf{h}_t^\top$ and $\mathbf{w}_t^\top$, the lower triangle of the joint observation covariance $\mathbf{G}$ is the lower triangle of $\mathbf{HW}^\top$, with r_t added to its diagonal. Symmetry supplies the upper triangle.

A.4 The LDL Factorization Produces Kalman Innovations

Let the centered observation vector for coordinate i be

$$\tilde{\mathbf{v}}_i = [\tilde{v}_{1,i}, \dots, \tilde{v}_{C,i}]^\top, \quad \text{Cov}(\tilde{\mathbf{v}}_i) = \mathbf{G}. \quad (43)$$

Because $r_t > 0$, $\mathbf{G}$ is positive definite. It therefore has the unique factorization

$$\mathbf{G} = \mathbf{LRL}^\top, \quad (44)$$

where $\mathbf{L}$ is unit lower triangular and $\mathbf{R}$ is positive diagonal. Define

$$\boldsymbol{\xi}_i := \mathbf{L}^{-1} \tilde{\mathbf{v}}_i. \quad (45)$$

Then

$$\text{Cov}(\boldsymbol{\xi}_i) = \mathbf{L}^{-1} \mathbf{GL}^{-\top} = \mathbf{R}. \quad (46)$$

Thus $\xi_{t,i}$ are mutually uncorrelated innovations and $R_{tt} = \text{Var}(\xi_{t,i})$. Since $\mathbf{L}^{-1}$ is lower triangular, $\xi_{t,i}$ depends only on observations through position t . It is exactly the innovation obtained by sequentially conditioning $v_{t,i}$ on $v_{1,i}, \dots, v_{t-1,i}$.

Let

$$\mathbf{E} := \mathbf{L}^{-1} \mathbf{W}, \quad (47)$$

and let $\mathbf{e}_t^\top$ be row t of $\mathbf{E}$. The covariance between the state at position t and its innovation is

$$\begin{aligned} \text{Cov}(\mathbf{x}_t, \xi_{t,i}) &= \sum_{s=1}^t (\mathbf{L}^{-1})_{ts} \text{Cov}(\mathbf{x}_t, v_{s,i}) \\ &= \sum_{s=1}^t (\mathbf{L}^{-1})_{ts} (\mathbf{f}_t \odot \mathbf{w}_s) \\ &= \mathbf{f}_t \odot \left(\sum_{s=1}^t (\mathbf{L}^{-1})_{ts} \mathbf{w}_s \right) \\ &= \mathbf{f}_t \odot \mathbf{e}_t. \end{aligned} \quad (48)$$

The Gaussian conditioning coefficient for innovation $\xi_{t,i}$ is therefore

$$\kappa_t = \frac{\text{Cov}(\mathbf{x}_t, \xi_{t,i})}{\text{Var}(\xi_{t,i})} = \frac{\mathbf{f}_t \odot \mathbf{e}_t}{R_{tt}}. \quad (49)$$

The sequential Kalman filter is precisely the innovations algorithm for this linear-Gaussian model. Hence this coefficient equals

$$\kappa_t = \frac{\hat{\mathbf{P}}_t \mathbf{k}_t}{r_t + \mathbf{k}_t^\top \hat{\mathbf{P}}_t \mathbf{k}_t} \quad (50)$$

from the tokenwise ExactKLA recurrence.

A.5 Exact Dense Covariance at the Next Chunk Boundary

At position C , the covariance with every innovation can be stacked as

$$\begin{aligned}\text{Cov}(\mathbf{x}_C, \boldsymbol{\xi}_i) &= [\mathbf{f}_C \odot \mathbf{e}_1 \quad \cdots \quad \mathbf{f}_C \odot \mathbf{e}_C] \\ &= \text{diag}(\mathbf{f}_C) \mathbf{E}^\top.\end{aligned}\tag{51}$$

Conditioning a jointly Gaussian state on the full innovation vector gives

$$\begin{aligned}\mathbf{P}_C &= \boldsymbol{\Sigma}_C - \text{Cov}(\mathbf{x}_C, \boldsymbol{\xi}_i) \text{Cov}(\boldsymbol{\xi}_i)^{-1} \text{Cov}(\boldsymbol{\xi}_i, \mathbf{x}_C) \\ &= \boldsymbol{\Sigma}_C - \text{diag}(\mathbf{f}_C) \mathbf{E}^\top \mathbf{R}^{-1} \mathbf{E} \text{diag}(\mathbf{f}_C).\end{aligned}\tag{52}$$

This is the same posterior covariance obtained after C sequential Riccati updates because conditioning a joint Gaussian distribution sequentially or jointly gives the same posterior.

A.6 Equivalence Across the Full Sequence

The first chunk begins with the same prior $\mathbf{P}_0 = \mu^{-1} \mathbf{I}$ as tokenwise ExactKLA. The derivation above shows that it produces the same gain at every token and the same posterior covariance at its boundary. Using that covariance as the prior for the next chunk repeats the same argument. Induction over chunks therefore proves that the chunk algorithm and tokenwise ExactKLA produce identical gains $\boldsymbol{\kappa}_1, \dots, \boldsymbol{\kappa}_T$ and identical boundary covariances. Applying these gains to the ExactKLA memory recurrence consequently produces the same $\mathbf{S}_t$ and reads $\mathbf{o}_t$.

A.7 Recovering LDL Factors from Cholesky

An implementation may use a Cholesky factorization

$$\mathbf{G} = \mathbf{C} \mathbf{C}^\top\tag{53}$$

instead of a direct $\text{LDL}^\top$ routine. Let

$$\mathbf{J} := \text{diag}(C_{11}, \dots, C_{CC}).\tag{54}$$

Then

$$\mathbf{L} = \mathbf{C} \mathbf{J}^{-1}, \quad \mathbf{R} = \mathbf{J}^2\tag{55}$$

gives a unit-lower-triangular $\mathbf{L}$ and satisfies

$$\mathbf{L} \mathbf{R} \mathbf{L}^\top = \mathbf{C} \mathbf{J}^{-1} \mathbf{J}^2 \mathbf{J}^{-1} \mathbf{C}^\top = \mathbf{C} \mathbf{C}^\top = \mathbf{G}.\tag{56}$$

In components, $R_{tt} = C_{tt}^2$ and $L_{ts} = C_{ts}/C_{ss}$, exactly as used in the proposed innovation derivation. Let $\mathbf{Y} = \mathbf{C}^{-1} \mathbf{W}$. Since $\mathbf{L}^{-1} = \mathbf{J} \mathbf{C}^{-1}$,

$$\mathbf{E} = \mathbf{L}^{-1} \mathbf{W} = \mathbf{J} \mathbf{Y}.\tag{57}$$

Therefore row t satisfies $\mathbf{e}_t = C_{tt} \mathbf{y}_t$, and

$$\frac{\mathbf{f}_t \odot \mathbf{e}_t}{R_{tt}} = \frac{\mathbf{f}_t \odot \mathbf{y}_t}{C_{tt}}.\tag{58}$$

Similarly,

$$\mathbf{E}^\top \mathbf{R}^{-1} \mathbf{E} = \mathbf{Y}^\top \mathbf{J} \mathbf{J}^{-2} \mathbf{J} \mathbf{Y} = \mathbf{Y}^\top \mathbf{Y}.\tag{59}$$

This proves the direct Cholesky formulas used by the proposed implementation and avoids explicitly forming the unit-lower factor or squaring and subsequently dividing by small Cholesky pivots.